

EduGenie: Google Gemini Powered Learning Assistant

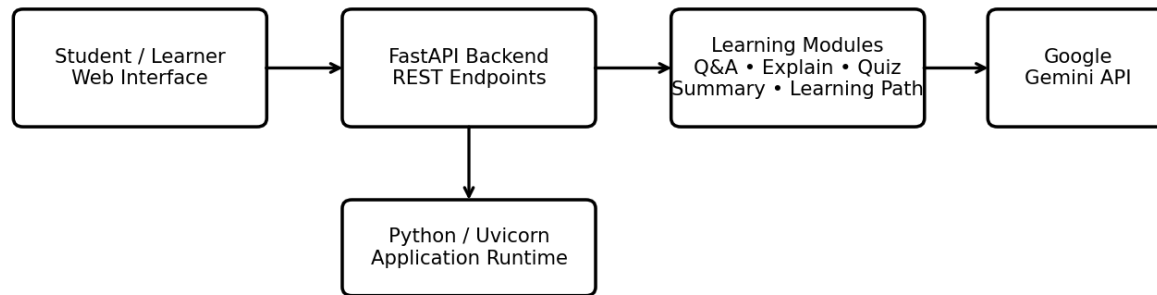
PROJECT DOCUMENTATION

AI-Powered Educational Assistant

Project ID: SWTID-2026-1393

Backend: FastAPI | Frontend: HTML + CSS + JavaScript | AI: Google Gemini

EduGenie - Technical Architecture



Repository: <https://github.com/bala712/EduGenie>

Hosted application: <https://edugenie-t3j2.onrender.com>

1. Project Description

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative AI. It provides a web-based interface where students can ask questions, request explanations, generate quizzes, summarize study material, and receive structured learning-path guidance.

The application uses a FastAPI backend to receive and process requests and a simple HTML/CSS/JavaScript frontend to present learning tasks and results. Google Gemini provides the main generative AI capabilities used by the learning functions.

EduGenie is intended to make common study activities faster and easier by bringing several learning-support functions into a single interface.

Core scenario:

A student enters a topic or question, selects the required learning task, submits it through the web interface, and receives an AI-generated response. The result is then displayed in the browser for study or revision.

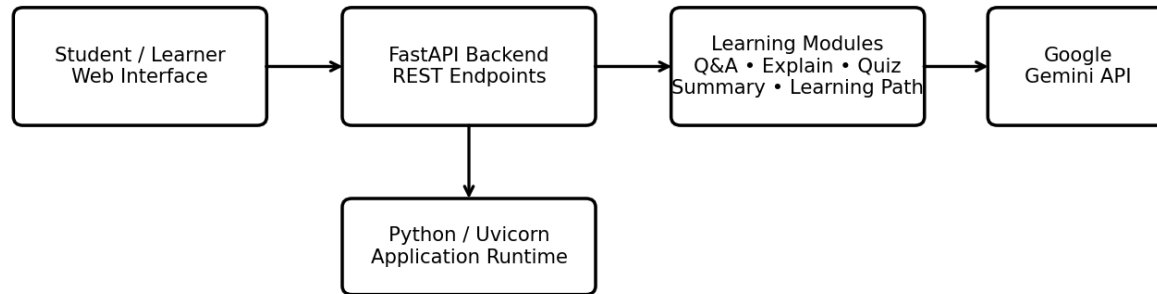
2. Scenarios

Scenario	Description
Scenario 1 – Question & Answer	A student enters an academic question and requests a concise answer. The frontend sends the request to FastAPI, which invokes the Q&A function and returns the generated answer.
Scenario 2 – Explanation	A student enters a difficult concept and requests a simpler explanation. The explanation module produces student-friendly learning content.
Scenario 3 – Quiz Practice	A student enters a topic and requests a practice quiz. The quiz module generates multiple-choice questions and presents them for self-assessment.
Scenario 4 – Summary	A student provides a long passage and requests a concise summary containing the important information for revision.
Scenario 5 – Learning Path	A learner provides a topic and receives structured guidance from beginner-level concepts toward more advanced learning.

3. Technical Architecture

EduGenie follows a frontend–backend architecture. The browser provides the user interface, FastAPI handles application requests, and the learning modules connect the request to the appropriate AI processing function.

EduGenie - Technical Architecture



Architecture flow: User → Web Interface → FastAPI REST API → Learning Module → Google Gemini → Generated Result → Web Interface.

A persistent database is not required by the current implementation, so the architecture focuses on application processing and the external AI service.

4. Prerequisites and Technologies

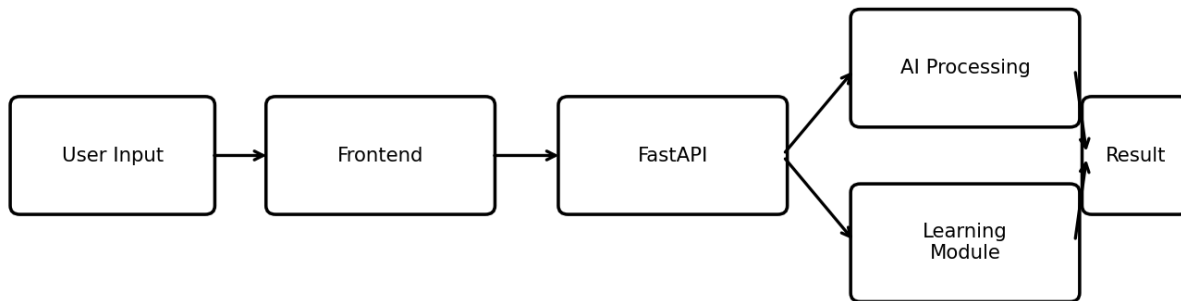
Technology / Requirement	Purpose
Python 3.10+	Application development and backend runtime.
FastAPI	REST API framework used to handle backend requests.
HTML5 / CSS3 / JavaScript	Web interface, styling and client-side interaction.
Google Gemini API	Generative AI for Q&A, explanations, quizzes, summaries and learning guidance.
Uvicorn	ASGI server used to run the FastAPI application locally.
Jinja2	HTML templating support where used by the frontend.
Git / GitHub	Source-code version control and repository hosting.
Render	Hosted deployment platform for the deployed EduGenie application.

Security requirement:

The Gemini API key should be stored as an environment variable and should not be committed to the public repository.

5. Project Workflow

EduGenie - Project Workflow



Milestone 1: Requirements and architecture – identify the learning tasks and define the frontend, FastAPI and Gemini interaction.

Milestone 2: Backend development – implement API routes and connect each route to the corresponding learning module.

Milestone 3: AI integration – configure Gemini access and construct prompts for each educational function.

Milestone 4: Frontend development – build the learning interface, inputs, task selection and result areas.

Milestone 5: Testing – verify Q&A, explanation, quiz, summary and learning-path functions.

Milestone 6: Deployment – publish the project through the configured Render deployment and maintain the GitHub source repository.

6. Development Environment Setup

The project can be prepared locally using Python and a virtual environment.

- Install Python 3.10 or later and verify the installation with `python --version`.
- Open the EduGenie repository/project folder in VS Code.
- Create and activate a virtual environment if required.
- Install the project dependencies using the requirements file.
- Create the environment configuration and provide the required Gemini API key.
- Start the FastAPI application using the configured Uvicorn command.
- Open the local application URL in a browser and test the learning functions.

Typical local start command: `uvicorn main:app --reload`

Typical local URL: `http://127.0.0.1:8000`

7. Core Functionalities Development

Module	Functionality
Student Learning Interface	Accepts topics/questions/text and displays AI-generated learning results.
AI Explanation	Produces clear explanations for submitted topics or questions.
Question & Answer	Answers student questions and provides clarification.
Learning Path	Provides structured learning guidance based on the selected topic.
Quiz Generation	Generates practice questions for self-assessment.
Summary / Study Notes	Converts longer educational text into concise revision content.
Gemini Integration	Connects the application to Google Gemini for generative AI output.
Backend API Processing	Receives frontend requests and routes them to the appropriate module.

8. FastAPI Backend Development

FastAPI acts as the application layer between the browser and the AI modules. Each learning task is exposed through a backend route or handler. The backend validates incoming data, invokes the requested learning function, and returns the generated result to the frontend.

Documented endpoint set:

Endpoint	Purpose
/qa	Question and answer processing.
/explain	Concept explanation processing.
/quiz	Quiz generation.
/summarize	Text summarization.
/learn/recommendations	Learning-path / recommendation generation.

Backend flow: Frontend request → FastAPI route → corresponding learning module → Gemini processing → structured response → frontend.

9. AI Integration and Prompt Handling

Google Gemini is used as the main generative AI service for EduGenie. The backend sends task-specific prompts and receives generated educational content.

The application separates the learning task from the presentation layer so that the frontend does not directly manage AI credentials or model requests.

- Q&A prompts request a direct and useful educational answer.
- Explanation prompts request simplified, learner-friendly explanations.
- Quiz prompts request practice questions with structured answer information.
- Summary prompts request concise revision material while retaining important points.
- Learning-path prompts request ordered guidance from foundational concepts toward advanced topics.

API-key handling:

The Gemini API key is configured through environment variables. The key should remain outside source control.

10. Explanation and Q&A Modules

Explanation Module

The explanation function is designed to convert difficult concepts into clearer, easier-to-understand learning content. The intended audience includes students and self-learners.

Question & Answer Module

The Q&A function accepts an educational question and returns a contextual answer. The frontend presents the response in the result area so that the learner can continue studying without leaving the application.

Expected interaction:

Step	Action
1	Student enters a question or topic.
2	Frontend sends the request to FastAPI.
3	FastAPI invokes the selected learning function.
4	Gemini generates the response.
5	Backend returns the result.
6	Frontend displays the answer/explanation.

11. Quiz Generation Module

The Quiz module is designed for active learning and self-assessment. A topic is supplied by the learner and the backend requests practice questions from the AI service.

The existing project documentation describes the quiz feature as generating three multiple-choice questions with four options each and structuring the result for display.

- Topic-based quiz generation.
- Multiple-choice question format.
- Four options for each question.
- Answer checking / self-assessment behaviour.
- AI-generated content returned through the FastAPI backend.

Quiz workflow: Topic → FastAPI /quiz → Gemini → Structured quiz result → Frontend → Student attempts quiz.

12. Summary and Learning Path Modules

Summary / Study Notes

The Summary module converts longer educational passages into concise revision material while retaining important information. This supports quick review before examinations or study sessions.

Learning Path

The Learning Path module provides structured guidance from beginner-level concepts toward more advanced learning. The documented design can include learning resources such as videos, articles or books when generated by the application.

Module	Input	Output
Summary	Long educational passage	Concise study summary
Learning Path	Topic / learning need	Ordered learning guidance

13. Frontend Development

The frontend provides a single learning interface for the main EduGenie functions. Users select or access a task, enter the required content, submit the request, and view the generated response.

- Task selection for Explain, Q&A, Quiz, Summary and Recommend Path.
- Text input for questions, topics or passages.
- Action/submit controls.
- Result display areas for generated content.
- Responsive styling for different screen sizes.
- Frontend-to-FastAPI request integration.

[EduGenie Home / Learning Interface Screenshot – insert actual project screenshot here]

14. User Interaction and Result Display

The frontend sends requests to the FastAPI backend and displays the returned result below the relevant learning task. This keeps the interaction simple: input → generate → review.

The interface is intended to support repeated learning tasks during the same session.

[EduGenie Q&A / Explanation Result Screenshot – insert actual project screenshot here]
--

For quiz generation, the result area presents the generated questions and options so the learner can attempt them and check understanding.

[EduGenie Quiz Result Screenshot – insert actual project screenshot here]

15. Local Testing and Verification

Functional testing focuses on verifying that each documented learning function accepts input, reaches the correct backend handler, receives AI output and displays the result.

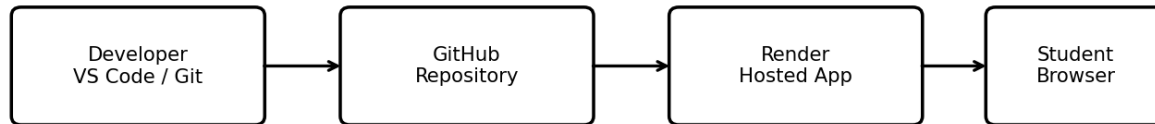
Test Case	Expected Result	Status
Ask a question	Answer displayed in result area	Verified in development workflow
Request explanation	Simplified explanation displayed	Verified in development workflow
Generate quiz	Multiple-choice quiz displayed	Verified in development workflow
Summarize passage	Concise summary displayed	Verified in development workflow
Request learning path	Structured learning guidance displayed	Verified in development workflow

For performance evidence, the separate Performance Testing document contains the project-specific test template and sample JMeter-style evidence.

16. Deployment

EduGenie has a GitHub repository and a deployed Render URL for project access.

EduGenie - Deployment Flow



Deployment Item	Details
Project ID	SWTID-2026-1393
GitHub Repository	https://github.com/bala712/EduGenie
Hosted Application	https://edugenie-t3j2.onrender.com
Hosting Provider	Render

Deployment flow: Developer → GitHub repository → Render deployment → Student browser.

17. Deployment and Run Instructions

Local execution

- Clone or open the EduGenie GitHub repository.
- Open the project in VS Code.
- Create/activate the Python virtual environment.
- Install dependencies from requirements.txt.
- Configure the Gemini API key through environment variables.
- Run the FastAPI application with Uvicorn.
- Open the local browser URL and test the modules.

Hosted execution

- Open the deployed Render URL in a browser.
- Use the learning interface to select a task.
- Enter the topic/question/passage.
- Submit the request and review the generated result.

Repository: <https://github.com/bala712/EduGenie>

Hosted URL: <https://edugenie-t3j2.onrender.com>

18. Challenges, Limitations and Security

Area	Challenge / Limitation	Handling
AI accuracy	Generated content can require human review.	Use AI as learning support and verify important information.
External service	Gemini availability and response time depend on the external API.	Handle API errors and maintain valid configuration.
API key security	A public repository must not contain secret keys.	Store credentials in environment variables and exclude .env from Git.
UI compatibility	Different screen sizes may require responsive styling.	Use responsive HTML/CSS design.
Personalization	Learning recommendations depend on the quality of user input.	Provide clear topic and learning requirements.

These limitations describe practical considerations for the current implementation; they do not represent unsupported claims about future system behaviour.

19. Future Scope

- Voice-based interaction for hands-free learning.
- Multilingual learning support.
- Mobile application and offline-oriented features.
- Progress dashboards and gamification.
- Teacher and parent dashboards.
- Integration with learning platforms such as Moodle or Google Classroom.
- Image/PDF input for doubt solving and summarization.
- Expanded analytics for learning progress.

Future scope is based on the existing EduGenie project documentation and is intended as possible expansion rather than currently implemented functionality.

20. Conclusion

EduGenie combines generative AI with a lightweight web interface to support personalized learning. Its FastAPI backend separates the main learning functions into explanation, question answering, quiz generation, summarization and learning-path support.

The project demonstrates how Google Gemini can be integrated into an educational application while keeping the frontend and backend responsibilities separated. The current deployment provides a GitHub source repository and a Render-hosted application.

The project can be extended through voice interaction, multilingual support, mobile access, progress analytics, collaborative learning features and richer educational inputs.

Project Details

Project Name	EduGenie – AI Learning Assistant
Project ID	SWTID-2026-1393
Repository	https://github.com/bala712/EduGenie
Hosted Application	https://edugenie-t3j2.onrender.com
Backend	FastAPI / Python
Frontend	HTML / CSS / JavaScript
AI	Google Gemini

End of Documentation